

■ Text Summarizer Study Notes

Fine-Tuning T5 for Dialogue Summarization (HuggingFace Transformers)

■ Topics Covered

- Loading & Exploring the SAMSum Dataset
- Text Cleaning with Regular Expressions (re)
- T5 Tokenizer & Tokenization (padding, truncation)
- T5ForConditionalGeneration Model
- Device Selection (CPU / CUDA / MPS)
- TrainingArguments & the Trainer API
- Model Training, Saving & Reloading
- Generating Summaries with Beam Search

Based on `Text_summarizer.ipynb` — T5-small fine-tuned on SAMSum dataset

SECTION 1 — Imports & Loading the Dataset

■ What is This Project About?

This notebook builds a **dialogue summarizer** — a model that reads a conversation (like a chat between two people) and produces a short summary of what was discussed. It uses the **SAMSum dataset** (a famous dataset of chat conversations with human-written summaries) and fine-tunes a pre-trained **T5 model** to do this task.

■ What is T5?

T5 stands for 'Text-To-Text Transfer Transformer'. It is a model created by Google that treats every NLP problem as a text-to-text problem — meaning you give it text as input, and it generates text as output. Summarization, translation, and question-answering can all be done using the same model architecture, just trained differently. **T5-small** is a smaller, faster version of T5, good for learning and limited compute.

■ What is HuggingFace Transformers?

transformers is a Python library by HuggingFace that provides ready-to-use pre-trained models (like T5, BERT, GPT) along with tools to train, fine-tune, and use them easily. Instead of building a neural network from scratch, you can load a model that already understands language and adapt it to your specific task.

■ Code from the Notebook:

```
import pandas as pd from transformers import T5Tokenizer, Trainer,
TrainingArguments, T5ForConditionalGeneration
```

import pandas as pd — pandas is used to read and manipulate tabular data (CSV files).

T5Tokenizer — converts raw text into numbers (token IDs) that the model can understand.

Trainer — a HuggingFace class that handles the entire training loop for you automatically.

TrainingArguments — a class to configure training settings (epochs, batch size, etc.).

T5ForConditionalGeneration — the actual T5 model class used for text generation tasks like summarization (as opposed to T5Model, which is the base model without a generation head).

■ Loading the Dataset:

```
train_data = pd.read_csv("/content/samsum-train.csv") val_data =
pd.read_csv("/content/samsum-validation.csv") train_data.head() train_data.shape
val_data.shape
```

pd.read_csv(path) — reads a CSV file into a pandas DataFrame (a table-like structure with rows and columns).

train_data — used to train the model.

val_data — used to check the model's performance during training, on data it has not learned from.

.head() — shows the first 5 rows, useful to peek at the data.

.shape — returns (number_of_rows, number_of_columns).

■ Dataset Columns Explained:

Column	Meaning
dialogue	The full chat conversation (input text)
summary	The human-written short summary (target/label text)

■ Sampling a Smaller Subset:

```
train_data = train_data.sample(n=4000, random_state=42).reset_index(drop=True)
val_data = val_data.sample(n=500, random_state=42).reset_index(drop=True)
```

.sample(n=4000) — randomly picks 4000 rows out of the full dataset. This is done to reduce training time (the full dataset may have 14,000+ rows).

random_state=42 — a 'seed' value that makes the random selection reproducible — running the code again gives the exact same 4000 rows.

.reset_index(drop=True) — after sampling, row index numbers become messy (e.g., 5, 1923, 47...). This resets them back to 0, 1, 2, 3... and drop=True means the old messy index is discarded, not kept as a new column.

■ Why sample only 4000 rows? Training on the full dataset takes much longer and needs more compute (GPU/RAM). For learning purposes, a smaller subset trains faster while still teaching the model the summarization pattern.

■ Your Own Easy Example:

```
import pandas as pd
df = pd.DataFrame({"x": range(100)})
small_df = df.sample(n=10, random_state=1).reset_index(drop=True)
print(small_df.shape) # (10, 1)
```

```
(10, 1)
```

■ Common Mistakes:

■ Forgetting `reset_index(drop=True)` — leads to confusing, non-sequential row indices later.

■ Sampling more rows than exist in the dataset ($n > \text{total rows}$) causes an error.

■ Not setting `random_state` — results become non-reproducible (different rows each run).

SECTION 2 — Data Pre-processing (Text Cleaning)

■ Why Clean Text Before Training?

Raw text data is messy — it may contain extra line breaks, multiple spaces, leftover HTML tags, and inconsistent capitalization. Feeding messy text directly to a model can confuse it and hurt performance. Cleaning standardizes the text so the model learns patterns more easily.

■ What is the 're' Module?

re is Python's built-in module for **Regular Expressions** (regex) — a mini-language for finding and replacing patterns in text. For example, you can match 'all digits', 'all whitespace', or 'anything inside angle brackets'.

■ Code: The Cleaning Function:

```
import re
def clean_data(text):
    text = re.sub(r"\r\n", " ", text) # lines
    text = re.sub(r"\s+", " ", text) # spaces
    text = re.sub(r"<.*?>", "", text) # html tags
    text = text.strip().lower()
    return text
```

■ Line-by-Line Explanation:

re.sub(pattern, replacement, text) — searches for 'pattern' inside 'text' and replaces every match with 'replacement'. Returns the new modified text.

- **re.sub(r"\r\n", " ", text)** — replaces Windows-style line breaks (\r\n, carriage return + newline) with a single space.
- **re.sub(r"\s+", " ", text)** — \s+ means "one or more whitespace characters" (spaces, tabs, newlines). This collapses multiple spaces/tabs into a single space.
- **re.sub(r"<.*?>", "", text)** — removes HTML tags like <p> or <h1>. The .*? means "match anything, but as few characters as possible" (non-greedy match) between < and >.
- **text.strip().lower()** — .strip() removes leading/trailing spaces; .lower() converts all text to lowercase so "Hello" and "hello" are treated the same.

■ Regex Symbols Cheat Sheet:

Symbol	Meaning	Example Match
\s	Any whitespace (space, tab, newline)	' ', '\t', '\n'
+	One or more of the previous symbol	'\s+' matches ' '
.	Any single character	matches 'a', '1', '#'
*	Zero or more of the previous symbol	'.*' matches anything
?	Makes the match non-greedy (shortest match)	'.*?' stops at first '>'
\r\n	Carriage return + newline (Windows line break)	line endings

■ Applying the Function to the Dataset:

```
train_data["dialogue"] = train_data["dialogue"].apply(clean_data)
train_data["summary"] = train_data["summary"].apply(clean_data)
val_data["dialogue"] = val_data["dialogue"].apply(clean_data) val_data["summary"]
= val_data["summary"].apply(clean_data)
```

.apply(clean_data) — runs the `clean_data()` function on every single row/value in that column. This is much faster and cleaner than writing a manual for-loop.

■ Your Own Easy Example:

```
import re def clean_data(text): text = re.sub(r"\s+", " ", text) text =
re.sub(r"<.*?>", " ", text) return text.strip().lower() sample = " Hello World!!
\n\n How are YOU? " print(clean_data(sample))
```

```
hello world!! how are you?
```

■ Common Mistakes:

■ Forgetting the raw string prefix 'r' before regex patterns — without it, `\s` and `\r\n` may be misinterpreted.

■ Using `.*` (greedy) instead of `.*?` (non-greedy) for HTML tags — this can accidentally delete text between two unrelated tags.

■ Applying `.lower()` before checking for proper nouns needed elsewhere in the pipeline — make sure lowercase is acceptable for your task.

SECTION 3 — Tokenization with T5Tokenizer

■ What is Tokenization (for Transformers)?

Neural networks cannot understand raw text — they only understand numbers. **Tokenization** is the process of converting text into a sequence of numbers called **token IDs**. Each ID corresponds to a word, sub-word, or character piece that the model recognizes from its vocabulary (built during the model's original pre-training).

■ Why a Specific Tokenizer (T5Tokenizer)?

Every pre-trained model has its own tokenizer that matches exactly how it was originally trained — using a different tokenizer would produce token IDs the model does not understand correctly. **T5Tokenizer.from_pretrained('t5-small')** downloads/loads the exact vocabulary and rules used when T5-small was trained.

■ Code: Loading the Tokenizer:

```
tokenizer = T5Tokenizer.from_pretrained("t5-small")
```

.from_pretrained('t5-small') — a HuggingFace convention used everywhere. It downloads (or loads from cache) all the files needed for the 't5-small' tokenizer.

■ Code: The Tokenize Function:

```
def tokenize(data): input = tokenizer(data["dialogue"], padding="max_length",
max_length=520, truncation=True) target = tokenizer(data["summary"],
padding="max_length", max_length=150, truncation=True) input["labels"] =
target["input_ids"] return input
```

■ Parameter-by-Parameter Explanation:

Parameter	Meaning
padding='max_length'	Pads (adds filler tokens) every sequence up to max_length, so all sequences in a batch are the same length
max_length=520 / 150	The maximum number of tokens allowed. Dialogue gets 520 (longer input), summary gets 150 (shorter input)
truncation=True	If text is longer than max_length, cut it off instead of causing an error

input['labels'] = target['input_ids'] — this is a key step! In sequence-to-sequence training, the model needs to know the 'correct answer' (target) for each input. 'labels' is the special key name that HuggingFace's Trainer looks for — it represents the token IDs of the summary, which the model tries to learn to predict.

(Note: the variable name 'traget' is a typo for 'target' in the original code — it still works fine in Python since it's just a variable name, but watch for typos like this!)

■ Think of it like a flashcard: 'input_ids' is the question (the dialogue) and 'labels' is the answer (the summary). During training, the model is repeatedly shown the question and learns to produce the answer.

■ Applying Tokenization to the Whole Dataset:

```
train_dataset = train_data.apply(tokenize, axis=1).tolist() val_dataset =  
val_data.apply(tokenize, axis=1).tolist()
```

.apply(tokenize, axis=1) — axis=1 means apply the function row-by-row (not column-by-column). Each row (a dialogue + summary pair) is passed into tokenize().

.tolist() — converts the resulting pandas Series into a plain Python list, which is the format the Trainer expects.

■ Inspecting One Tokenized Example:

```
train_dataset[0]
```

```
{'input_ids': [363, 19, 8, ...], 'attention_mask': [1, 1, 1, ..., 0, 0], 'labels':  
[566, 19, 3, ...]}
```

input_ids — the token IDs for the cleaned dialogue.

attention_mask — a list of 1s and 0s. 1 means 'this is a real token, pay attention to it'; 0 means 'this is just padding, ignore it'.

labels — the token IDs for the target summary.

■ Your Own Easy Example:

```
from transformers import T5Tokenizer tokenizer =  
T5Tokenizer.from_pretrained("t5-small") result = tokenizer("hello world",  
padding="max_length", max_length=8, truncation=True) print(result)
```

```
{'input_ids': [21820, 296, 1, 0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 0, 0, 0,  
0, 0]}
```

Notice the 0s at the end — those are padding tokens added to reach max_length=8, and the attention_mask correctly marks them as 0 (ignore).

■ Common Mistakes:

■ Using a different max_length for input during training vs. when generating summaries later — keep input max_length consistent (520 vs. 512 mismatch can cause subtle issues).

■ Forgetting truncation=True — long sequences will throw errors instead of being cut off.

■ Naming the labels key something other than 'labels' — the Trainer specifically looks for this exact key name.

SECTION 4 — Loading the Model & Selecting a Device

■ What is T5ForConditionalGeneration?

This is the T5 model with a 'generation head' attached — meaning it's specifically set up to generate text output (not just understand text, but produce new sequences). It uses an **encoder-decoder architecture**: the encoder reads and understands the input dialogue, and the decoder generates the summary word by word.

■ Code: Loading the Pre-trained Model:

```
model = T5ForConditionalGeneration.from_pretrained("t5-small")
```

This loads T5-small with all its pre-trained weights — meaning it already understands general English language patterns from its original training. We will **fine-tune** it (continue training it) specifically on our summarization examples.

■ What is a 'Device' in Deep Learning?

Training neural networks involves huge amounts of matrix math. A **GPU (Graphics Processing Unit)** can do this math far faster than a regular **CPU**. PyTorch lets you choose which hardware device runs your computations.

Device	What it Means	When Available
cpu	Regular processor — slow for deep learning	Always available, fallback option
cuda	NVIDIA GPU acceleration	On machines with NVIDIA GPUs (e.g. Colab GPU runtime)
mps	Apple Metal Performance Shaders (GPU)	On Apple Silicon Macs (M1/M2/M3 chips)

■ Code: Automatic Device Selection:

```
import torch if torch.backends.mps.is_available(): device = torch.device("mps") elif torch.cuda.is_available(): device = torch.device("cuda") else: device = torch.device("cpu") print("device: ", device) model.to(device)
```

torch.backends.mps.is_available() — checks if an Apple GPU is available.

torch.cuda.is_available() — checks if an NVIDIA GPU is available.

torch.device("...") — creates a device object representing where computations should run.

model.to(device) — physically moves the model's weights to that device's memory (GPU memory or CPU RAM). This is required before training/inference on that device.

This if/elif/else checks devices in priority order: MPS → CUDA → CPU, picking the fastest one available on the current machine.

■ Expected Output (example on Google Colab with GPU):

```
device: cuda
```

■ Your Own Easy Example:

```
import torch x = torch.tensor([1.0, 2.0, 3.0]) print(x.device) # cpu by default if
torch.cuda.is_available(): x = x.to("cuda") print(x.device)
```

■ Common Mistakes:

■ Moving the model to a device but forgetting to move the input tensors too — this causes a 'tensors on different devices' error.

■ Assuming GPU is always available — always check with `is_available()` rather than hardcoding 'cuda'.

SECTION 5 — TrainingArguments & the Trainer

■ What is TrainingArguments?

TrainingArguments is a configuration object — it does not train anything itself, it just stores all the settings (hyperparameters) that control how training will happen. Think of it like a settings panel you fill out before starting training.

■ Code: Setting Up TrainingArguments:

```
train_args = TrainingArguments( output_dir="./result_of_model", num_train_epochs=6,
weight_decay=0.01, per_device_train_batch_size=8, per_device_eval_batch_size=8,
eval_strategy="epoch", save_strategy="epoch", warmup_steps=500 )
```

■ Parameter-by-Parameter Explanation:

Parameter	Meaning
output_dir	Folder where model checkpoints and logs are saved
num_train_epochs=6	The model sees the entire training dataset 6 full times
weight_decay=0.01	A regularization technique that slightly shrinks weights to prevent overfitting
per_device_train_batch_size=8	Number of examples processed together in one training step, per device
per_device_eval_batch_size=8	Same idea, but for evaluation (validation) steps
eval_strategy='epoch'	Run evaluation on val_data once after every full epoch
save_strategy='epoch'	Save a model checkpoint once after every full epoch
warmup_steps=500	Gradually increase the learning rate for the first 500 steps instead of starting at full speed (helps

■ What is an 'epoch'? One epoch = one complete pass through the entire training dataset. If you have 4000 training examples and batch_size=8, one epoch = 500 training steps ($4000 \div 8 = 500$ steps).

■ What is the Trainer Class?

Trainer is a HuggingFace class that automates the entire training process: it handles the training loop (forward pass, loss calculation, backward pass, optimizer step), evaluation, checkpoint saving, logging, and more — so you don't have to write that code manually.

■ Code: Creating and Running the Trainer:

```
trainer = Trainer( model=model, args=train_args, train_dataset=train_dataset,
eval_dataset=val_dataset ) trainer.train()
```

model=model — the T5 model object to be trained.

args=train_args — the TrainingArguments object created earlier.

train_dataset / eval_dataset — the tokenized lists prepared in Section 3.

trainer.train() — this single line kicks off the ENTIRE training process — running for 6 epochs,

evaluating after each epoch, and saving checkpoints automatically.

■ What Happens During `trainer.train()` (Conceptually):

1. Take a batch of 8 examples from `train_dataset`
2. Feed the `input_ids` into the model, get predicted output
3. Compare prediction with the true 'labels' using a loss function
4. Calculate gradients (how wrong each weight was) via backpropagation
5. Update the model's weights to reduce the error slightly
6. Repeat for all batches, for all 6 epochs
7. After each epoch, evaluate on `val_dataset` and save a checkpoint

■ Saving and Reloading the Fine-tuned Model:

```
model.save_pretrained("./saved_summary_model")
tokenizer.save_pretrained("./saved_summary_model") # Later, reload them: model =
T5ForConditionalGeneration.from_pretrained("./saved_summary_model") tokenizer =
T5Tokenizer.from_pretrained("./saved_summary_model")
```

`.save_pretrained(path)` — saves the model's weights and configuration (and the tokenizer's vocabulary/settings) to a local folder, so you don't need to retrain every time.

`.from_pretrained(path)` — this same method works for loading either an original HuggingFace model name (like 't5-small') OR a local folder path — very convenient!

■ Common Mistakes:

■ Setting `batch_size` too high for available GPU memory — causes 'CUDA out of memory' errors.

■ Forgetting to save the tokenizer along with the model — without it, reloading the model alone is not enough to process new text correctly.

■ Too many epochs on a small dataset can cause overfitting (model memorizes training data instead of generalizing).

SECTION 6 — Generating Summaries (Inference)

■ What is 'Inference'?

Inference means using the already-trained model to make predictions on new, unseen data — as opposed to **training**, where the model learns from labeled examples. Here, inference means: give the model a new dialogue, and it generates a summary.

■ Code: The Full Summarization Function:

```
def summarize_dialogue(dialogue): dialogue = clean_data(dialogue) # clean #
tokenize inputs = tokenizer( dialogue, padding="max_length", max_length=512,
truncation=True, return_tensors="pt" ).to(device) # generate the summary => token
ids model.to(device) targets = model.generate( input_ids=inputs["input_ids"],
attention_mask=inputs["attention_mask"], max_length=150, num_beams=4,
early_stopping=True ) # decoded our output summary = tokenizer.decode(targets[0],
skip_special_tokens=True) return summary
```

■ Step-by-Step Breakdown:

Step 1 — Clean: `dialogue = clean_data(dialogue)` — applies the same cleaning function from Section 2, so new input text matches the format the model was trained on.

Step 2 — Tokenize: Converts the cleaned text into token IDs. `return_tensors='pt'` is new here — 'pt' means return PyTorch tensors (instead of plain Python lists), which is what the model expects as input. `.to(device)` moves these input tensors to the same device (GPU/CPU) as the model.

Step 3 — Generate: `model.generate()` runs the model's decoder to actually produce new text, token by token, rather than just doing a single forward pass.

Step 4 — Decode: `tokenizer.decode()` converts the generated token IDs back into human-readable text.

■ Understanding `model.generate()` Parameters:

Parameter	Meaning
<code>input_ids</code>	The tokenized dialogue (what to summarize)
<code>attention_mask</code>	Tells the model which tokens are real vs. padding (ignore padding)
<code>max_length=150</code>	Generate a summary of at most 150 tokens
<code>num_beams=4</code>	Use Beam Search with 4 beams (explained below) for better quality output
<code>early_stopping=True</code>	Stop generating once all beams reach an end-of-sequence token

■ What is Beam Search (`num_beams=4`)?

When generating text word by word, the model could simply pick the single most likely next word every time (called 'greedy decoding') — but this often leads to repetitive or low-quality results. **Beam search** instead keeps track of the top 4 most promising partial sentences ('beams') at each step, and

only at the very end picks the overall best complete sentence. This usually produces noticeably better summaries than greedy decoding.

■ Analogy: Imagine writing a sentence one word at a time, but instead of committing to your first guess at every word, you keep 4 different draft sentences alive simultaneously and only choose the best-sounding one once they're all finished.

■ What is `skip_special_tokens=True`?

Tokenizers add special marker tokens during processing — like `<pad>`, `</s>` (end-of-sequence), or `<unk>` (unknown word). These are needed internally by the model but should not appear in the final human-readable summary. `skip_special_tokens=True` removes them automatically when decoding.

■ Code: Testing the Function on a Sample Dialogue:

```
test_dialogue = """ Reporter: In today's technology news, artificial intelligence
continues to expand... Expert: AI systems are becoming more capable due to advances
in deep learning... ... (multi-line conversation) ... """ summary =
summarize_dialogue(test_dialogue) print("Summary: ", summary)
```

Summary: ai is expanding rapidly across industries. experts discuss concerns about bias, explainability and the need for responsible ai development and regulation.

(Note: actual output will vary depending on how well the model trained — this is an illustrative example of the expected style of output.)

■ Your Own Easy Example (using a pre-trained model without fine-tuning, for practice):

```
from transformers import T5Tokenizer, T5ForConditionalGeneration tok =
T5Tokenizer.from_pretrained("t5-small") mdl =
T5ForConditionalGeneration.from_pretrained("t5-small") text = "summarize: The cat
sat on the mat and then went outside to play in the garden." inputs = tok(text,
return_tensors="pt") out = mdl.generate(inputs["input_ids"], max_length=20,
num_beams=2) print(tok.decode(out[0], skip_special_tokens=True))
```

the cat sat on the mat

Note: the base (non-fine-tuned) T5 model expects a task prefix like 'summarize: ' before the text to know which task to perform — this is part of how T5's text-to-text format works.

■ Common Mistakes:

■ Forgetting `return_tensors='pt'` during inference — the model expects PyTorch tensors, not plain lists, when calling `.generate()`.

■ Not moving the model AND the inputs to the same device — mismatched devices cause runtime errors.

■ Setting `num_beams=1` and expecting beam search behavior — `num_beams=1` is actually equivalent to greedy decoding.

■ Forgetting `skip_special_tokens=True` — output will contain ugly tokens like `<pad>` or `</s>`.

SECTION 7 — Full Program Flow Summary

How the Entire Summarizer Pipeline Works, Step by Step:

Step 1 Load & sample data

Read SAMSum train/validation CSVs with pandas, then randomly sample a smaller subset (4000 train, 500 val) to keep training fast.

Step 2 Clean the text

Apply `clean_data()` (regex-based) to remove line breaks, extra spaces, HTML tags, and normalize casing for both dialogue and summary columns.

Step 3 Tokenize

Use `T5Tokenizer` to convert cleaned dialogue (max 520 tokens) and summary (max 150 tokens) into token IDs, with the summary's IDs stored as 'labels'.

Step 4 Load model & pick device

Load `T5ForConditionalGeneration` ('t5-small'), detect the best available device (MPS/CUDA/CPU), and move the model onto it.

Step 5 Configure & run training

Define `TrainingArguments` (epochs, batch size, warmup, eval/save strategy), then create a `Trainer` and call `trainer.train()` to fine-tune the model.

Step 6 Save the fine-tuned model

Save both model and tokenizer to disk with `save_pretrained()`, so they can be reloaded later without retraining.

Step 7 Generate summaries (inference)

Write `summarize_dialogue()`: `clean` → `tokenize` → `model.generate()` with beam search → decode tokens back into readable text.

Quick Reference — Key Functions/Classes Cheat Sheet:

Function / Class	Purpose
<code>pd.read_csv()</code>	Load a CSV file into a DataFrame
<code>df.sample(n, random_state)</code>	Randomly select n rows reproducibly
<code>re.sub(pattern, repl, text)</code>	Find-and-replace text using regex

<code>T5Tokenizer.from_pretrained()</code>	Load the tokenizer matching a pre-trained T5 model
<code>tokenizer(text, padding, max_length, truncation)</code>	Convert text into token IDs + attention mask
<code>T5ForConditionalGeneration.from_pretrained()</code>	Load the T5 model for text generation tasks
<code>torch.cuda.is_available()</code>	Check if an NVIDIA GPU is available
<code>model.to(device)</code>	Move model weights to CPU/GPU memory
<code>TrainingArguments(...)</code>	Configure training hyperparameters
<code>Trainer(model, args, train_dataset, eval_dataset)</code>	Automate the full training loop
<code>trainer.train()</code>	Starts fine-tuning the model
<code>model.save_pretrained(path)</code>	Save model weights/config to disk
<code>model.generate(...)</code>	Produce new text using the trained model
<code>tokenizer.decode(ids, skip_special_tokens)</code>	Convert token IDs back into readable text

■ Big Picture: Why Each Stage Matters

- **Cleaning** removes noise so the model focuses on actual language patterns, not formatting artifacts.
- **Tokenization** bridges the gap between human text and the numeric world neural networks operate in.
- **Fine-tuning** takes a general-purpose pre-trained model and specializes it for one specific task (dialogue summarization) using a relatively small amount of labeled data.
- **Beam search generation** balances quality and computation when producing the final summary text.